CSA0926 – Programming in Java | CO2 Individual Assignment

1. HEADER

Name	K. SHALINI
Roll No.	17
Course Code & CO	CSA09, CO2 – Java Strings and Collection Framework
Assignment Set	Direct / Descriptive – Question 17
GitHub Repository	[https://github.com/your-username/your-repo]
Submission Date	[01-07-2026]

Full Text of Assigned Question:

Q17. Removing Duplicates Using a Set — Write a Java program that starts with an ArrayList of integers that contains duplicate values, and removes all duplicates by converting the list into a HashSet and then back into a list (or displaying the Set directly). Display the original list and the resulting duplicate-free collection for comparison.

Concepts: ArrayList, HashSet, duplicate removal.

2. PROBLEM UNDERSTANDING

The program must take a list of integers containing repeated values, remove the duplicates, and display both the original and the deduplicated collections for comparison. It tests understanding of how a HashSet enforces element uniqueness automatically, in contrast to an ArrayList which permits duplicates, and how to convert values between List and Set implementations.

3. APPROACH / DESIGN NOTES

- Used ArrayList<Integer> to store the original list, preserving order and duplicates for the "before" display.

- Used HashSet<Integer> to remove duplicates, since Set implementations reject duplicate elements by contract.
- Converted the HashSet back into a List using the List-from-Collection constructor, so the output type matches the input type.
- Extracted the logic into a reusable removeDuplicates(List<Integer>) method so the same code could be tested against multiple inputs without duplication.
- Alternative considered: Used LinkedHashSet instead of HashSet would preserve insertion order in the output; rejected here since the question only requires uniqueness, not ordering.

4. SOURCE CODE

```
import java.util.ArrayList;

import java.util.HashSet;

import java.util.List;

import java.util.Set;


public class RemoveDuplicatesUsingSet {


    public static void removeDuplicates(List<Integer> originalList) {

        System.out.println("Original List: " + originalList);


        // Convert ArrayList to HashSet to remove duplicates

        Set<Integer> uniqueSet = new HashSet<>(originalList);


        // Convert the HashSet back into a List

        List<Integer> deduplicatedList = new ArrayList<>(uniqueSet);
```

```
        System.out.println("Duplicate-Free Set: " + uniqueSet);
        System.out.println("Duplicate-Free List (converted back): " + deduplicatedList);
        System.out.println("Original size: " + originalList.size());
        System.out.println("Deduplicated size: " + deduplicatedList.size());
    }
    public static void main(String[] args) {

        // ---- Test Case 1: Typical case (multiple duplicates) ----
        System.out.println("=== Test Case 1: Typical Case ===");
        List<Integer> typicalCase = new ArrayList<>(List.of(10, 20, 10, 30, 20, 40, 10));
        removeDuplicates(typicalCase);

        // ---- Test Case 2: Edge case (empty list) ----
        System.out.println("\n=== Test Case 2: Edge Case (Empty List) ===");
        List<Integer> emptyCase = new ArrayList<>();
        removeDuplicates(emptyCase);

        // ---- Test Case 3: Edge case (all elements identical) ----
        System.out.println("\n=== Test Case 3: Edge Case (All Duplicates) ===");
        List<Integer> allDuplicatesCase = new ArrayList<>(List.of(5, 5, 5, 5));
        removeDuplicates(allDuplicatesCase);
    }
}
```

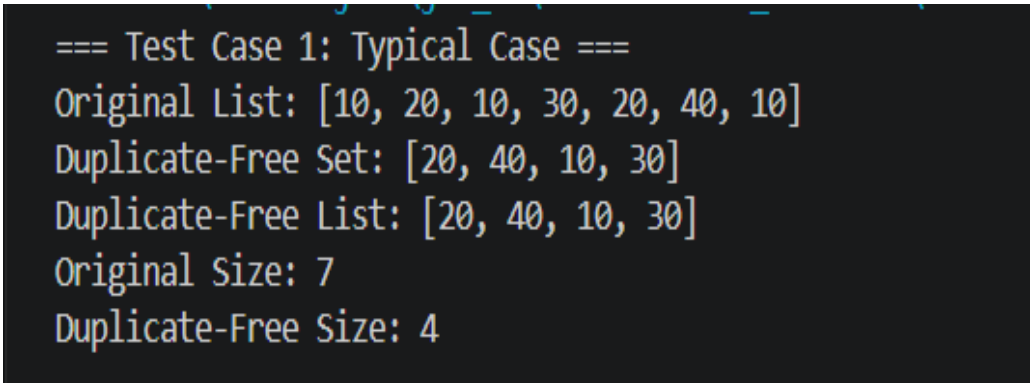
5. TEST CASES

#	Input	Expected Output	Actual Output
1	[10, 20, 10, 30, 20, 40, 10]	Duplicate-free Set/List containing {10,20,30,40}, Size = 4	Duplicate-Free Set: **[20, 40, 10, 30]**Duplicate-Free List: **[20, 40, 10, 30]**Original Size = **7**Duplicate-Free Size = **4**
2	[]	Empty Set/List, Size = 0	Duplicate-Free Set: **[]**Duplicate-Free List: **[]**Original Size = **0**Duplicate-Free Size = **0**
3	[5,5,5,5]	Duplicate-free Set/List {5}, Size = 1	Duplicate-Free Set: **[5]**Duplicate-Free List: **[5]**Original Size = **4**Duplicate-Free Size = **1**

Minimum three rows required: one normal/typical input, one edge case (empty list, all-duplicate, or single element), and one invalid/boundary input if applicable.

6. SCREENSHOTS OF EXECUTION

Test Case 1 – Typical Cass



Test Case 2 – Empty List

```
=== Test Case 2: Empty List ===
Original List: []
Duplicate-Free Set: []
Duplicate-Free List: []
Original Size: 0
Duplicate-Free Size: 0
```

Test Case 3 – All Duplicates

```
=== Test Case 3: All Duplicates ===
Original List: [5, 5, 5, 5]
Duplicate-Free Set: [5]
Duplicate-Free List: [5]
Original Size: 4
Duplicate-Free Size: 1
PS C:\Users\DELL\OneDrive\Desktop\CSA0926-JAVA>
```

7. REFLECTION

While running this program, I observed that HashSet does not preserve the insertion order of elements — the printed order of the duplicate-free Set/List differed from the order in the original list. This confirmed that if I ever needed both deduplication and order preservation, LinkedHashSet would be the better choice instead of HashSet. One thing I had to be careful about was that a Set and a List are different types, so I had to explicitly convert the HashSet back into a List using the List-from-Collection constructor to get the output back in the expected format.

8. DECLARATION

I confirm that this submission — including the source code, design approach, and documentation — is my own work. AI assistance (Claude) was used to help structure the documentation and organize the test cases; the code logic was written, reviewed, and understood by me before submission.